

GALAXY: Understanding Variations in Solutions to Data Science Problems at Scale

ANONYMOUS AUTHOR(S)**

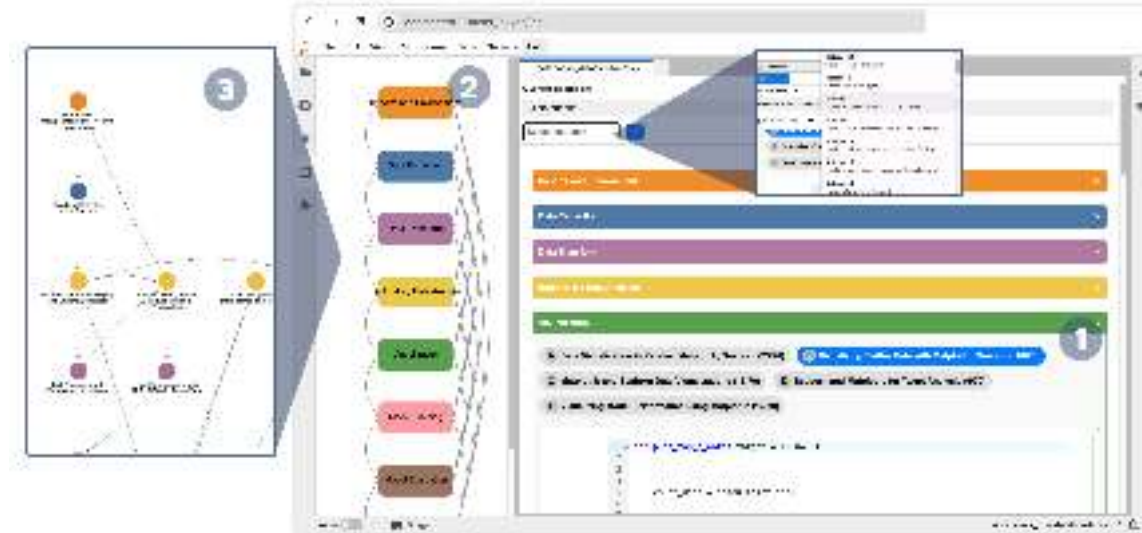


Fig. 1. Interface components of GALAXY. 1) Notebook view : Users can select the notebooks, stages and approaches to see. 2) Galactic view: Graph of the transitions between stages. 3) Stellar view: Graph of the transitions between clusters.

Understanding problem-solving approaches in computational notebooks is challenging due to the non-standard, nonlinear, and exploratory nature of coding, and this challenge increases when understanding multiple notebooks simultaneously. Notebook authors often experiment with diverse strategies, making it difficult to derive qualitative patterns and insights across multiple computational notebooks. In this paper, we present GALAXY, a Jupyter notebook extension designed to simplify the reading and comparison of data science solutions, reducing the mental effort required to interpret code at scale. GALAXY classifies code cells by data science stages and uses clustering techniques to distinguish different methodological approaches across notebooks, providing both visual and textual representations. We evaluated the effectiveness of GALAXY through a within-subject study with N=20 notebook users. We also discussed how this approach can be applied in various educational contexts, helping learners benefit from peer solutions in online communities, and enabling instructors to gain insights from students' assignments.

Additional Key Words and Phrases: JupyterLab extension, Workflow, NLP, Flowchart

ACM Reference Format:

Anonymous Author(s). 2025. GALAXY: Understanding Variations in Solutions to Data Science Problems at Scale. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

1 INTRODUCTION

Data scientists experiment with various machine learning models, feature engineering techniques, and hyper-parameter tuning strategies to identify the most effective solutions for a project. The diversity of approaches can complicate comparisons or assessments of individual methods, as it involves a range of algorithms, data preprocessing techniques, and workflows for model development and evaluation. Understanding the methods used across multiple notebooks is crucial in both academic and professional environments. For example, instructors grading student assignments need to review various submissions while maintaining an overall understanding. This task becomes more challenging with poorly structured code [10]. Similarly, in online knowledge communities like Kaggle, data science learners benefit from examining others' submissions in order to improve their own models and algorithms, but browsing dozens of similar submissions can be an exhausting process. Recruiters, too, must compare notebooks from candidates, searching not only for top performers but also for unique solutions that stand out, making it a time-consuming task. With the rise of large language model (LLM)-generated code, users also face the challenge of validating and understanding multiple potential solutions generated by AI [11, 37]. Despite the importance and generalizability of the task to review multiple notebook solutions for the same data science problem, the common approach is still to review each notebook individually.

Reading multiple code files written by individuals with varying skill levels and inconsistent conventions is a tenuous task. Unlike natural language, which is processed in specialized brain regions, code is interpreted by domain-general executive areas [19]. Deciphering poorly written code imposes significant mental strain on developers [10], potentially impairing their judgment after extended periods of analysis. The issue is equally prevalent when working with notebooks as opposed to traditional source code files. Sequentially reviewing numerous notebooks is an exhausting process, often resulting in cognitive overload. Focusing too closely on one notebook often makes it difficult to maintain a broader understanding of the entire solution space or to identify patterns across multiple submissions.

Several tools have been developed to address the cognitive load associated with reviewing multiple code files, particularly in educational contexts (e.g., grading students' coding exercises). Test cases only check if the code produces the correct output for specific inputs. They don't show how students approach the problem or how they structure their solutions. And instructors need the full picture to assess students' understanding, problem-solving strategies, and code quality beyond just correct outputs. Overcode [12] enables instructors to simultaneously view various student approaches by filtering for similarities in variable modifications. In contrast, CFlow [39] aids users in understanding the overall structure of code files, allowing navigation from high-level patterns to detailed implementations. Other tools, such as Codeopticon and Vizprog [14, 38], tackle the challenge of real-time understanding of student code during in-class exercises. However, those tools primarily focus on short and linear problems like algorithm and data structure problems. In a Machine Learning context, such filtering is not enough to capture the intricacies of a notebook. Problems are more exploratory, meaning two different notebooks may have the same end result, while having very different exploration paths.

To address this challenge, we introduce GALAXY, a tool which allows users to view multiple Jupyter notebooks at once and visualize their workflows using a Flowchart widget. Each notebook's code cells are classified by their data science stages and grouped into clusters based on the methodologies and algorithms employed. Each cluster is titled to highlight the relevant technologies and methodologies, giving to code a context in natural language. Our tool is intuitive, user-friendly, and allows users to visualize any subset of the notebook collection. With GALAXY, users can focus on specific data science stages and filter by the approaches taken in the notebooks. As illustrated in figure 1.1, our tool orders code cells by their respective data science stage. Users can select the notebooks they wish to view and click

on a cluster to see the associated code cells. Figures 1.2 and 1.3 display the flowchart of the selected notebooks. In the Galactic view, the nodes represent the stages, and in the Stellar view, they represent the clusters.

We validated GALAXY in a user study (N=20) in which participants were asked to answer questions based on notebooks from two datasets. They had to complete the task twice. Once without our tool, going through each notebook individually as the baseline. And once using GALAXY as the experimental condition. Participants reported a significant cognitive load when trying to read and understand code from multiple notebooks in the baseline condition. The study shows the effectiveness of our tool. Our tool allowed participants to answer questions faster and with less cognitive strain.

This work makes the following key contributions to the field of HCI :

- An interface to visualize code cells from different notebooks, organized by data science stage, cluster, and notebook, alongside a graph highlighting the flow and steps within a subset of notebooks.
- A pipeline classifying, clustering and titling notebook's code cells .
- A user study to evaluate how efficient GALAXY is in understanding notebooks compared to reading notebooks sequentially.

2 RELATED WORK

2.1 Understanding Student Submissions at Scale

When teaching Data Science, it is important for teachers to gain insights on how students think and apprehend the taught materials. Additionally, reviewing each code submission individually is a very time-consuming process. Clustering, visualizing student submission and allowing to filter them based on specific criteria can improve the evaluating workflow [12]. Prior work also mentions the issue of tracking students' progress in real-time during in-class exercises [38]. This addresses the issue of learners outnumbering teachers by far in DS classes. Some tools focus on scaling tutors' ability to assist learners [14]. Moreover, with the rise of MOOCs, it has become important to provide similar kinds of interactions online as in class with a teacher asking questions to students. Previous work demonstrated the effectiveness of interactive MOOCs and presenting the students' answers and feedback at scale [21].

2.2 Data Science Programming

Jupyter notebooks have become the favored format for data scientists to explore solutions for data-intensive problems. Previous work has highlighted how researchers have embraced Jupyter notebooks and the FAIR principles (Findable, Accessible, Interoperable, Reusable) they follow [29], despite the increased complexity compared to traditional source code files. Notebooks' cells, that can be executed in any order, make the sequence of execution less rigid than in traditional files [32]. Another benefit from Jupyter notebooks is the ability to alternate executable code and explanatory text, making it popular among data scientists and data science teachers [22, 30, 32]. However, previous work investigated the benefits and downsides of this format [5] among which was the issue of searching for code across multiple notebooks. While markdown cells are an essential component of notebooks, it was perceived by some as encouraging bad coding practices [26]. Furthermore, collaboration on a notebook for data science tasks is another issue brought forth [35]. GALAXY aims to introduce a new structure among notebooks to encourage a more organized workflow. Early articles acknowledged that DS code can be categorized into conceptual stages of a DS pipeline, due to its iterative and exploratory nature [8, 36]. Attempts had been done to provide a set of categories to organize DS code cells [7], but a standardized set is yet to be agreed upon. The Code4ML [9] dataset provides class labels for notebook code cells, organized in a 2-layered taxonomy tree, which serves as a foundation for GALAXY's classification labels. The first layer of the tree

would be : While democratizing, there are still many improvements that can be made to python notebooks to improve the teaching and workflow in Data Science, which is what GALAXY aims to do.

2.3 Code visualization

Flowcharts and code have always been closely intertwined. Algorithms can be represented as flowcharts, and flowcharts can be translated into algorithms. Previous research shows that flowcharts help students understand algorithms more effectively than simple pseudo code [31], even when dealing with algorithms of varying complexity. Given that novice programmers find flowcharts easier to understand than pseudo code, research has explored bypassing the traditional coding process altogether by allowing programmers to design flowcharts that represent their desired outcomes [4]. Flowcharts can not only generate code, but can also be generated from code. Creating flowcharts from code is a unique way to detect code similarity between source files from different programming languages [40]. This method offers a unique, language-agnostic way to analyze code, enhancing cross-language comparison and understanding. In an effort to make code easier to understand, GALAXY makes use of the benefits of flowcharts to highlight where notebooks are similar and pinpoint at which stages they diverge.

2.4 Code Classification and Clustering

Multiple ways to classify code have been explored in prior research. While GALAXY leverages the power of Machine Learning and Natural Language Processing to do this task, it is possible to achieve reasonable results using a Multinomial Naive Bayes to classify small code snippets [2]. As large language model (LLM) coding assistants, such as Github Copilot and ChatGPT, become more prevalent, code classification can benefit by distinguishing between human-written and machine-generated code [18]. For our purposes, which involve DS code, having datasets with annotated code snippets is necessary. Though there are a few [7, 9], their availability is still limited. Additionally, Machine Learning can provide deeper insights into code by clustering it. Semantic clustering for instance can be used to provide a useful first impression of an unfamiliar system [23]. Other research has explored clustering to help developers search source code more efficiently [17]. An interesting possibility with code clustering is obviously to group similar codes, to compare for plagiarism for instance [28] or to repair incorrect code [25]. In contrast, one can gain insights on code snippets by unveiling the different approaches taken to solve a task [3], which can be useful for scaling feedback or for research [20]. Classification and clustering can therefore help in gaining valuable insights on a large amount of code. Furthermore, the information gained from the code can be translated to natural language using code summarization [33], a method commonly used to evaluate the accuracy of LLMs [15]. Our work differs slightly as we make use of it for groups of similar code snippets to provide a broader context.

3 USAGE SCENARIOS

To explain the problem and guide the design, we describe four common usage scenarios that showcase the usefulness of our tool.

3.1 Instructors reviewing student submissions

As a data science instructor, Alex needs to review student submissions for a data science assignment. The assignment involves evaluating multiple Jupyter notebooks from students, all of which address the same issue but in different ways. Alex finds it unpleasant to switch between different notebooks; he loses sight of past solutions and finds it difficult to recognize patterns that are shared among the notebooks. Using GALAXY, Alex will be able to compare notebooks side

by side in an organized manner without having to open each one individually. He will be able to identify common and unique methods across notebooks rapidly through the cluster titles. Instead of only looking at one notebook at a time, he will be able to view multiple notebooks and focus on specific data science stages. Doing so, he will gain insights to the patterns and the outliers of the students submissions, making it easier for a targeted feedback.

3.2 Data science competitions

Bob just started studying Python and is enthusiastic about trying out his new skills. He recently found out about Kaggle competitions and wants to participate in one. Being new to the subject, he starts looking at the already submitted notebooks. However, popular competitions can have in the thousands of submissions [1, 6]. Bob feels overwhelmed by the vast number of notebooks shared by other users. He doesn't know where to start, what techniques to use, or which methods are most effective. Shifting through dozens of notebooks is confusing and time-consuming. With GALAXY, Bob can quickly go through a summary of all notebooks in the competition, categorized by the different stages of the data science process. This enables Bob to easily identify trends in the approaches competitors take, such as common preprocessing techniques or popular models for him to try out later.

3.3 Students working on a machine learning assignment

As a student working on a machine learning assignment, Casey frequently experiments with various approaches to improve her results, resulting in multiple versions of the same project using different techniques or libraries. Managing these versions and comparing their effectiveness becomes cumbersome and time-consuming, especially as the number of variations grows. Using GALAXY, Casey can easily review the different workflows, compare the approaches taken in each version, and identify which specific methods or configurations yielded better results. This saves time and helps Casey focus on optimizing her solution more effectively, rather than manually tracking each version of her work.

3.4 Recruiters assessing competition submissions

As a recruiter hiring for a machine learning role, Daniel is tasked with reviewing the submissions from a competition where hundreds of candidates have participated. Daniel wants to quickly identify candidates who approached the problem differently or demonstrated unique skills. Manually reviewing each submission would be a daunting and time-consuming process. With GALAXY, Daniel can efficiently filter the submissions by methodologies and technologies used, making it easier to find candidates who took innovative or distinct approaches. This efficient process saves time and allows Daniel to focus on the most relevant submissions, making it easier to identify standout candidates.

4 SYSTEM DESIGN

4.1 Design Requirements

When dealing with code cells, GALAXY aims to give viewers as much context as possible before they read any code. Users should be able to compare notebooks and have an overview of methodological trends. To exploit the underlying iterative structure of notebooks [36], GALAXY classifies code cells into distinct Data Science stages, inspired by the first layer code labeling of Drozdova et al. [9]. The stages are: Imports and Environment, Data Extraction, Data Transformation, Exploratory Data Analysis, Visualization, Model Training, Model Evaluation, Data Export. With this framework, GALAXY was developed based on the following key requirements:

- **Support viewers understand nuances:** Viewers should have a clear overview of the different methodologies used in the notebooks.
- **Natural reading experience:** To address the inertia of users experienced with reading Jupyter notebooks, GALAXY should keep a similar feeling for an intuitive use.
- **Minimizing Cognitive Load:** The UI should be simple and user-friendly. To avoid overwhelming the user, information should be displayed in an organized and visually comfortable way. Users should also have control over how much information is displayed.

By following these design requirements, GALAXY offers an efficient and user-friendly tool for Data Science professionals, enabling more effective exploration and comparison of notebook content.

4.2 Interface

In this section, we will go over the different components in our UI, shown in figure 1.

4.2.1 Notebook view. Code cells are spread across colored boxes, each representing one data science stage and titled accordingly (Fig. 2.1). This structure provides the first layer of textual context, as the viewer understands that the code is associated with the DS stage in which it is grouped. The colors serve to distinguish visually boxes from each other, creating clear limits in the viewer's eyes. The viewer can open or close the stage-box (Fig. 2.4), giving them control over the displayed information.

Inside each stage-box, code cells are organized in clusters, which are listed at the top the stage-box (Fig. 2.2), serving as the second layer of context. Reading the cluster titles reveals the different methodologies taken in the selected notebooks. Without having to look at the code, the viewer already has information regarding the various approaches used. Making the titles clickable also allows to filter cells by cluster, in order to focus on code related to a specific approach. Each title is also accompanied by the cluster's size, meaning the number of code cells associated with the cluster, giving an idea of the predominant clusters and the rarer ones. The goal was to favor a cluster-by-cluster exploration experience, as the viewer should be able to easily see and compare the different coding approaches taken.

As the view can easily become overwhelming with code cells coming from multiple notebooks, the viewer has the option to filter code cells by clicking on the notebook name displayed alongside each code cell (Fig. 2.3), or by selecting notebooks at the top of the main frame (Fig. 1). This allows the viewer to view one, a subset of notebooks, or all notebooks at the same time. Doing so, they also have the option to open a notebook in a new tab, if they desire to read the notebook the "standard way". Displaying data from multiple notebooks at once can easily lead to an overwhelming screen. Filtering is therefore an important feature as it gives the viewer the option to regulate the amount of information being displayed, and select notebooks that better align with their goals.

The choice to categorize cells by DS stages has the consequence that cells wouldn't be displayed in their original order, since the order of stages inside a notebook is variable (one might start with two cells for Data Extraction, then one cell for Visualization and then again a cell in Data Extraction). With this new interface design for reading code, it is important to make it feel as natural as possible. In fact, due to most people being used to Jupyter notebooks' vertical reading style, it was a conscious design choice to keep the reading flow vertical.

4.2.2 Galactic view. When the "All notebooks" option is selected, a vertical linear graph is displayed (Fig. 1.2), in which each rectangular node corresponds to a stage-box. Edges along the sides connect nodes between each other to show how solutions transition between stages. An edge from node A to node B represents a code cell from stage A being



Fig. 2. Stage-box. 1) Stage name: The name of the stage associated with the code cells. 2) Selected cluster: The cluster selected by the user to show only the associated cells. 3) Notebook ID: The unique ID of the notebook associated with the cell next to it. 4) Open/Close stage: A button to show or hide the stage-box. 5) Code cell from original notebook. 6) Cluster ID: The unique ID of the cluster associated with the cell next to it.

followed by a code cell from stage B. Such a graph serves to provide the viewer with the first layer of context for the set of notebooks, this time visually. It depicts a greater overview of the chronology of stages that are present across the multiple notebooks. As the purpose of this flowchart is to give the viewer a chronological and spacial overview, the nodes are made clickable, allowing the viewer to quickly navigate on the main frame between the different stage-boxes. The choice was made to display this view as a side bar on the left side, as it shouldn't take away the focus from the actual content of the notebooks. Displaying it along the notebook view allows the viewer to have the visual representation of the DS flows in parallel to the actual code, aiming for an easier understanding of the structure of the notebooks.

4.2.3 Stellar view. When selecting a subset of all the notebooks (not all), a vertical flow-graph is displayed (Fig. 1.3). Each layer of the graph corresponds to a stage-box and each node in one layer (colored accordingly to the stage-boxes color) corresponds to a cluster of that stage-box. Edges are drawn from node A to node B if in one of the selected notebooks, there is a code cell from cluster A followed by a code cell from cluster B. This flowchart represents visually both layers of context, emphasizing more the second, as the cluster titles are written along each node. Along the same lines, the nodes are also made clickable. Upon clicking on a node, the notebook view moves to the associated stage-box and adds a filter for the corresponding cluster. This is designed to encourage a cluster-by-cluster exploration, while allowing the user to visualize in detail the flow of the notebooks. For the same reasons as the Galactic view, it is displayed in the left sidebar

4.3 Backend pipeline

The logic behind GALAXY states that Data Science code can be split into a fixed set of stages [36], which can be common across notebooks. Additionally, for each stage, various approaches can be taken to perform the same task.

4.3.1 Code cells classification into data science stages. The first step in GALAXY consists of submitting a list of .ipynb notebook files or the name of a Kaggle competition from which to pull notebooks. Using OpenAI's GPT-4o, every code cell of each submitted notebook is classified into a data science stage (Fig. 3.1). The choice for these classes and for their labels has been refined using our classification accuracy on the Code4ML dataset [9], achieving approx. 75% accuracy.

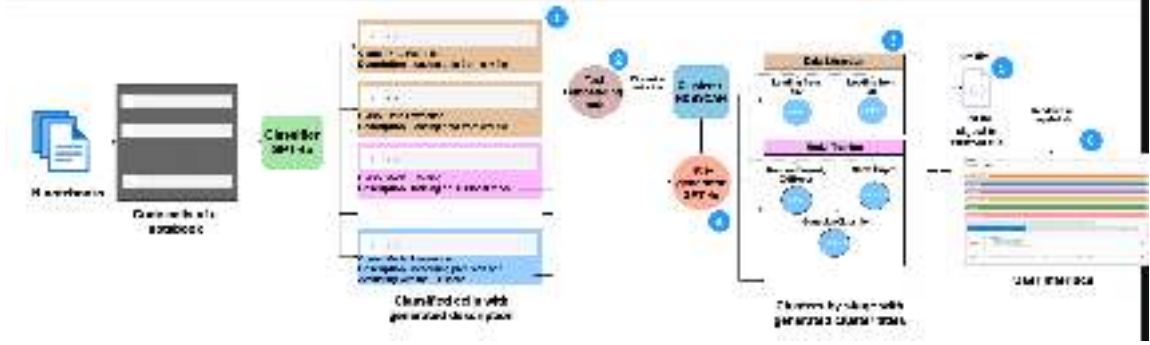


Fig. 3. Diagram of GALAXY’s backend pipeline. A set of notebooks is given as input and each is first classified into a class alongside a generated description (1). Descriptions are embedded and their dimension is reduced (2). Among each class, embedded vectors are clustered (3). A title is generated for each cluster using the associated natural language description of each clustered vector (4). Data is summarized into a .viz file (5) and is then rendered on the UI (6).

Since more context improves code cell classification, we include markdown cells, previously classified cells and their predicted classes in the message history. Alongside the predicted class, we also generate a plain-text description of the code cell which will be used later on in the clustering step.

4.3.2 Code cell clustering. Once classified, within each stage, the classified cells are clustered using an unsupervised approach to identify the different methods used in that stage (Fig. 3.3). To this end, we create vector embeddings from the previously generated plain-text description of the code cells (Fig. 3.2). Then, we reduce the dimensions of the vector embeddings to 2 components. Finally, we cluster a first time with a minimal cluster size of 10 samples and minimal neighbor count of 2 the reduced embeddings [24]. Outliers are clustered a second time independently.

Metric	Value
Silhouette Score	0.19
Calinski-Harabasz Index (CH-index)	105
Davies-Bouldin Index	1.08

Table 1. Clustering Evaluation Metrics by taking the average score over notebooks from the *NLP with Disaster Tweets* Kaggle competition.

4.3.3 Cluster titling. The last step is to generate titles for each cluster (Fig. 3.4). This step is important as it will provide the necessary context for the viewer to understand the code directly from the cluster title (Fig 2.2). Moreover, it categorizes differently written but similar code under a shared natural language description. This makes it easier to remember similarities between notebooks rather than comparing code blocks with each other. These titles are again generated using OpenAI’s GPT-4o model by inputting the list of code descriptions of each cluster. The model is instructed to keep the titles short to a 5 word maximum and to explicitly mention the methods and technologies used inside the cluster. Previously generated cluster titles are kept in the message history, so that the model generates a title as distinct as possible from the other ones.

4.4 Implementation

The backend pipeline was coded entirely in Python making use of OpenAI’s GPT-4o model for classification, description- and title-generation. Vector-embedding was performed using the OpenAI’s text-embedding-ada-002 model. Dimension reduction on the vectors was done using TSNE [34] and clustering was performed using the HDBSCAN [24] classifier. For faster processing, .ipynb-file processing and classification were parallelized. All necessary information for the UI display is kept in a custom .viz file format (Fig. 3.5), which is a JSON object standardized for GALAXY. GALAXY was developed as a Jupyter Lab extension. The UI was designed using CSS and the ReactJS framework. The processing is entirely based on the provided .viz file format.

5 EVALUATION

To evaluate the effectiveness of GALAXY in reducing cognitive load for notebook reviewers, we conducted a lab study with 20 participants. This study is approved by the ethics commission.

Participants were given the task of reviewing notebooks from two Kaggle competitions, once using GALAXY and once the standard way serving as a baseline method. We aimed to evaluate the following aspects:

- (1) *How does GALAXY help in reducing cognitive load when reviewing multiple notebooks?* The goal of GALAXY is to decrease cognitive load during when reviewing multiple notebooks. To evaluate this, we administer a post-task survey using questions adapted from the NASA TLX index [16]. Participants will provide feedback on how effective they find the tool for completing specific tasks and managing mental demand.
- (2) *How does GALAXY grant participants more insights on the approaches taken in the notebooks?* Do participants perform better (in terms of accuracy, speed, and overall task efficiency) when using GALAXY than with the baseline method.
- (3) *How do participants use GALAXY to their benefits?* This question explores participants’ intentions to use GALAXY in their future academic or professional tasks, along with feedback on areas for improvement.

5.1 Recruitment

The 20 participants were recruited via our local university forum and through personal connections. Participants registered for the experiment via a dedicated online form. All participants were aged between 21 and 25 and were enrolled in various master’s programs, including data science, computer science, robotics, and mechanical engineering. A prerequisite for participation was prior experience with Python notebooks and familiarity with Kaggle competitions or similar data science platforms, ensuring that participants had the necessary background for the study.

5.2 Datasets

We chose Kaggle competitions to source the notebooks based on their suitability for beginners, coverage of diverse Machine Learning areas, and allowance for multiple solution approaches. With those requirements in mind, we decided to go with two popular Kaggle competitions: *d* (A) and *Natural Language Processing with Disaster Tweets* (B) [1, 6]. We extracted 20 notebooks per competition, to ensure meaningful clustering and to cover many different approaches. The study was able to cover the tool’s effectiveness from basic machine learning to NLP techniques.

5.3 Study Conditions

For the evaluation, we aimed to minimize bias from fatigue and familiarity with reading notebooks. The standard way of reviewing notebooks served as the baseline condition, and using our tool as the experimental condition. This setup allowed us to assess the within-subjects effects of using GALAXY across different tasks and datasets. By switching datasets between iterations, we prevented participants from being influenced by prior knowledge of the competitions. Additionally, participants' perception of the tool may differ depending on whether they used it in the first or second iteration.

5.4 Study procedure

As mentioned before, participants need to fulfill two tasks in two separate iterations. Each task consists of analyzing a set of notebooks from one of the two Kaggle competitions and answering five competition-specific questions about the methodological trends within the given notebooks. They have 20 minutes to answer the five questions in each iteration. At the end of a task, participants fill out a 5-question survey adapted from NASA TLX [16] to assess their cognitive load, by assigning a score from 1 to 5 for the following criteria: mental demand, hurriedness, performance, effort and frustration.

We host the GALAXY extension, the original notebooks of both competitions, and the .viz file generated from the competitions on a customized JupyterHub. All studies were conducted online via Zoom, allowing participants to share their screen with researchers for better input. At the start of each study, we offered a tutorial and provide participants with a 5-minute warm up example. They then proceeded with either the baseline or experiment, switching competitions and methods after 20 minutes. At the end of the study, participants fill out a 3-minute post-evaluation survey in which they can give their feedback on the GALAXY extension. During this last step, participants were asked to argument on their given feedback vocally leading to an informal interview to further discuss their feelings about the extension.

5.5 Data collection and Analysis

We collected from several sources, including the interview transcriptions, task questionnaires, screen recordings, and observation notes. For the task questionnaire that consists of participants' responses to the task, and their post-task self-reported NASA TLX ratings, we use Wilcoxon Signed-rank test to compare the differences between the two conditions. We cross-referenced interview transcripts, observation notes, as well as screen recordings with quantitative data from questionnaires to gain a more comprehensive understanding of participants' preferences and experiences.

5.6 USER STUDY RESULTS

5.6.1 Quantitative Feedback.

The post-task self-reported NASA TLX ratings provided direct insight into the first research question: How does GALAXY help in reducing cognitive load when reviewing multiple notebooks compared to the baseline? Participants assessed their cognitive load across the baseline and experiment conditions, the results indicated several key differences. Participants rated the baseline method as requiring higher mental demand, effort, and frustration compared to the experiment method as shown in Figure 4. The Wilcoxon test results 3 support these observations. Participants experienced significantly lower ($p=0.025$) mental demand when using GALAXY ($M=3.20$, $SD=0.95$) compared to the baseline ($M=3.90$, $SD=0.72$). Similar trends were observed when evaluating frustration ($p=0.035$) and effort ($p=0.413$). However,

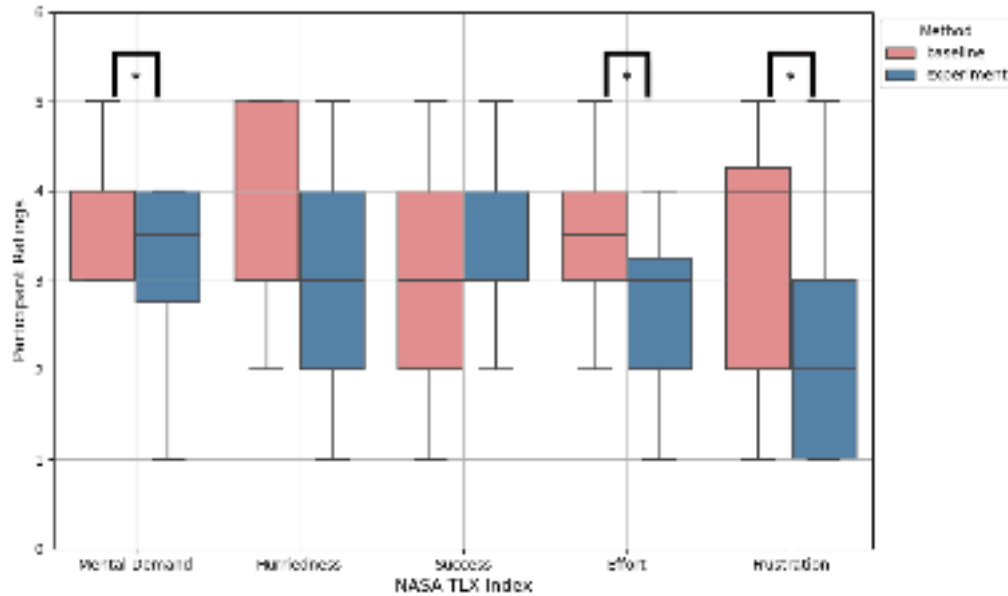


Fig. 4. Participants self-reported scores for NASA TLX questions. * indicate $p < 0.05$

no statistically significant difference was observed in terms of how much success they felt ($p=0.213$) or how hurried they were ($p=0.282$).

Dataset	p-value	Baseline Mean (Std)	Experiment Mean (Std)
titanic	0.985435	0.685 (0.165514)	0.675 (0.141886)
disaster	0.304204	0.590 (0.137267)	0.635 (0.107728)

Table 2. Wilcoxon Test Results for Titanic and Disaster Datasets by Aggregating the Score of Each Participant

The participants' answers to the competition offered valuable perspective on the second research question: How does GALAXY grant participants more insights on the approaches taken in the notebooks? We can observe in Table 2 that there is no significant change between the baseline and the experiment across competition ($p > 0.05$ for both competitions).

5.6.2 Qualitative Feedback.

In the post-task survey, the users ($N=20$) were asked to rate if GALAXY was more useful at completing the task than the baseline method. 16 participants found our tool more suitable than the baseline method. The reasons for this are made clear by the survey results and informal interviews, which we address next.

Finding 1: Ease of learning and intuitive interface: 16 participants explicitly mentioned that the extension was easy to learn and had an intuitive interface that facilitated quick on-boarding. The interface made it easy to visualize data and navigate through the tasks. P1 noted, "Very easy to use and a very clean layout, which makes it easy to visualize

the data and understand the structure”. This simplicity helped users transition from manual methods to using the tool efficiently. P6 said, “The tool is more straightforward than looking after the structure of each notebook manually”.

Finding 2: Effective tool features: 18 participants stated that the tool’s features effectively addressed their needs, with 9 specifically mentioning the Grouped Notebooks feature being their favorite. P3 shared, “Things were grouped together and easy to navigate” and P11 commented on how they liked how the tool grouped machine learning steps like evaluation and training into one place, reducing the need for manual search. Additionally, 12 participants highlighted Classification as a useful feature, and 6 appreciated the tool’s interactiveness. P6’s feedback emphasizes the usefulness of grouping notebooks: “*That the corresponding steps of the ML process (evaluation, training, etc.) were all at the same spot. I didn’t have to scroll and switch tabs.*” The Graph feature was mentioned by 3 participants as their favorite feature.

Finding 3: Responsiveness and performance satisfaction: All the participants appreciated the tool’s Responsiveness, with 19 noting minimal delays during use, including loading times and interaction delays. P13 highlighted the fact that the app was highly responsive. This positive feedback was a key factor in overall satisfaction, with 17 expressing satisfaction with the tool. These high ratings demonstrate that responsiveness and ease of use were critical elements in participants’ overall satisfaction and efficiency in completing tasks.

Finding 4: Suggestions for improving 10 participants suggested improvements to clustering and titling of code cells. P3 mentioned that “*naming of the cells is sometimes confusing*”. Additionally, 5 participants recommended extra features, such as improving the hierarchy clarity, a chatbot for searching approaches, or an abstract for each cluster. Other areas noted for enhancement included the graph (N=4) and ease of use (N=2). The users emphasized that clearer labels would improve the overall experience when navigating multiple notebooks.

Finding 5: Future use in academic and professional settings 18 participants mentioned that they would consider using the tool for future tasks, particularly in academic settings like grading or comparing submissions. P3 commented, “*I could see this tool being useful for teachers looking at student submissions in a general context*”. Additionally, several participants noted that it would be valuable in research, especially for quickly building on existing projects. P5 said, “*This tool would help researchers access information faster, allowing them to build on techniques quickly.*”

6 DISCUSSION

6.1 Improving current algorithms

The usability and effectiveness of GALAXY rely heavily on the accuracy of its classification, clustering, and titling algorithms. Just like in OverCode [12], key features of our tool are the grouping of similar solutions, and being able to filter by approaches. Accurate classification ensures that each cell is correctly assigned to the appropriate data science stage, minimizing the risk of errors. Any misclassification can result in cells being placed in incorrect stages, potentially confusing users and affecting the performance of the clustering algorithm. Moreover, the clustering algorithm needs to output a limited number of groups. Balancing clarity and complexity in the graph is essential: fewer clusters simplify the visualization, but may result in too generic titles.

This is why in the future we would like to keep upon improving the accuracy of our algorithms. A classification model specifically designed and trained on notebook code snippets could potentially be able to significantly improve the accuracy of our tool. However, there is no standardized taxonomy for Data Science stages and therefore no dataset with manually annotated code. Code4ML is a good contribution to this issue. However, most code snippets in the Code4ML dataset are automatically annotated [9], which comes with a certain degree of unreliability. Since classification errors directly impact clustering, improving it is the first crucial step to making GALAXY more useful.

6.2 Flowchart

Using GALAXY, users are able to capitalize on the usefulness of flowcharts in understanding code [31]. The Galactic view provides information on the transitions between different Data Science stages. However, it could be even more informative if it included additional details, such as the number of cells within each stage or the frequency of transitions between stages. This would allow users to better understand the areas of focus in the notebooks, whether it be data preprocessing, visualization, or model training.

Moreover, the Flowchart in the Stellar view can become overwhelming when dealing with very long notebooks or when too many are selected simultaneously. It could be beneficial to explore alternative visualization methods, reducing verticality or representing fewer clusters at a time. Additionally, adding information along the edges, similar to how machine learning pipelines are typically visualized [13], could enhance clarity and usability.

6.3 Design limits

The current design provides less information from each notebook compared to viewing them individually. Specifically, markdown content is omitted, even though it may be crucial for understanding the context of certain code cells. Usually, markdown cells are used for indicating what is done next in the notebook, providing additional context to for the subsequent cells [27]. The issue lies in deciding how many code cells should be linked each markdown cell, if they should be linked at all. Doing so could result in repeated markdown across different stages, potentially confusing the user. Additionally, questions arise about how to handle multiple consecutive markdown cells or how to represent the original spacing between code cells from the notebook.

Due to our current design decisions, cell outputs are not displayed in the view, although users can still access them by opening the original notebook using the dedicated button. It highlights the question of how important is the output of a notebook in understanding the approaches taken. We argue that most of the useful outputs are intended for the original writer of the notebook to understand the dataset or to compare outputs between two executions of the program [27]. While outputs include plotting and model scores, they are less important when focusing on the similarities and overall flow of a set of notebooks.

6.4 Limitation of study design

Although the study participants did not fully represent all potential users described in the usage scenarios, we observed a common learning trajectory. Initially, participants did not interact with GALAXY as intended, but they adapted quickly. By the end of the study, most participants were using the tool efficiently, indicating a shallow learning curve.

Additionally, to give participants an objective while evaluating GALAXY, we designed the questions to cover general aspects of the competition. Since they were designed as multiple-choice questions, participants could already have an idea of what they are looking for. For instance, some made use of the word-search function on web pages, going the opposite way of our tool being designed for exploratory purposes.

7 CONCLUSION

Our studies have demonstrated that gaining insights from reading jupyter notebooks is a lengthy and cognitively heavy task. In this paper, we introduced GALAXY, a tool designed to visualize notebooks in a new organized format, providing users with insights about the underlying methodological trends among multiple notebooks, and successfully

contributing to alleviating the cognitive load of our test users. However, further improvements of the classification and clustering techniques could unlock the full potential of the extension. Nonetheless, this paper makes a valuable contribution to understanding the complications involved in learning from python notebooks and how a new interface could be designed to address them.

REFERENCES

- [1] Phil Culliton Addison Howard, devrishi and Yufeng Guo. 2019. Natural Language Processing with Disaster Tweets. <https://kaggle.com/competitions/nlp-getting-started>
- [2] Kamel Alreshedy, Dhanush Dharmaretnam, Daniel M. German, Venkatesh Srinivasan, and T. Aaron Gulliver. 2018. SCC: Automatic Classification of Code Snippets. [arXiv:1809.07945 \[cs.SE\]](https://arxiv.org/abs/1809.07945) <https://arxiv.org/abs/1809.07945>
- [3] Elizaveta Artser, Anastasiia Birillo, Yaroslav Golubev, Maria Tigina, Hieke Keuning, Nikolay Vyahhi, and Timofey Bryksin. 2024. Clustering MOOC Programming Solutions to Diversify Their Presentation to Students. (03 2024).
- [4] Kanis Charntaweeekhun and Somkiat Wangsiripitak. 2006. Visual Programming using Flowchart. In *2006 International Symposium on Communications and Information Technologies*. 1062–1065. <https://doi.org/10.1109/ISCIT.2006.339940>
- [5] Souti Chattopadhyay, Ishita Prasad, Austin Z Henley, Anita Sarma, and Titus Barik. 2020. What’s wrong with computational notebooks? Pain points, needs, and design opportunities. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [6] Will Cukierski. 2012. Titanic - Machine Learning from Disaster. <https://kaggle.com/competitions/titanic>
- [7] Cristina Sarasua Dhivyabharathi Ramasamy, Alberto Bacchelli, and Abraham Bernstein. 2023. Workflow analysis of data science code in public GitHub repositories. *Empirical Software Engineering* 28, 1 (2023), 7. <https://doi.org/10.1007/s10664-022-10229-z> Epub 2022 Nov 19.
- [8] Pedro Domingos. 2012. A few useful things to know about machine learning. *Commun. ACM* 55, 10 (oct 2012), 78–87. <https://doi.org/10.1145/2347736.2347755>
- [9] Anastasia Drozdova, Ekaterina Trofimova, Polina Guseva, Anna Scherbakova, and Andrey Ustyuzhanin. 2023. Code4ML: a large-scale dataset of annotated Machine Learning code. *PeerJ Computer Science* 9 (2023), e1230.
- [10] Sarah Fakhoury, Yuzhan Ma, Venera Arnaoudova, and Olusola Adesope. 2018. The effect of poor source code lexicon and readability on developers’ cognitive load. In *Proceedings of the 26th Conference on Program Comprehension (Gothenburg, Sweden) (ICPC ’18)*. Association for Computing Machinery, New York, NY, USA, 286–296. <https://doi.org/10.1145/3196321.3196347>
- [11] Kasra Ferdowsi, Ruanqianqian (Lisa) Huang, Michael B. James, Nadia Polikarpova, and Sorin Lerner. 2024. Validating AI-Generated Code with Live Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI ’24)*. Association for Computing Machinery, New York, NY, USA, Article 143, 8 pages. <https://doi.org/10.1145/3613904.3642495>
- [12] Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Trans. Comput.-Hum. Interact.* 22, 2, Article 7 (mar 2015), 35 pages. <https://doi.org/10.1145/2699751>
- [13] Zhiqiang Gong, Ping Zhong, and Weidong Hu. 2019. Diversity in Machine Learning. *IEEE Access* 7 (2019), 64323–64350. <https://doi.org/10.1109/ACCESS.2019.2917620>
- [14] Philip J. Guo. 2015. Codeopticon: Real-Time, One-To-Many Human Tutoring for Computer Programming. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology (Charlotte, NC, USA) (UIST ’15)*. Association for Computing Machinery, New York, NY, USA, 599–608. <https://doi.org/10.1145/2807442.2807469>
- [15] Rajarshi Haldar and Julia Hockenmaier. 2024. Analyzing the performance of large language models on code summarization. *arXiv preprint arXiv:2404.08018* (2024).
- [16] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, 139–183. [https://doi.org/10.1016/S0166-4115\(08\)62386-9](https://doi.org/10.1016/S0166-4115(08)62386-9)
- [17] Marcus Hägglund, Francisco J. Peña, Sepideh Pashami, Ahmad Al-Shishtawy, and Amir H. Payberah. 2021. COCLUBERT: Clustering Machine Learning Source Code. In *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*. 151–158. <https://doi.org/10.1109/ICMLA52953.2021.00031>
- [18] Oseremen Joy Idialu, Noble Saji Mathews, Rungroj Maipradit, Joanne M. Atlee, and Mei Nagappan. 2024. Whodunit: Classifying Code as Human Authored or GPT-4 Generated - A case study on CodeChef problems. In *Proceedings of the 21st International Conference on Mining Software Repositories (MSR ’24)*. ACM, 394–406. <https://doi.org/10.1145/3643991.3644926>
- [19] Anna A Ivanova, Shashank Srikant, Yotaro Sueoka, Hope H Kean, Riva Dhamala, Una-May O’Reilly, Marina U Bers, and Evelina Fedorenko. 2020. Comprehension of computer code relies primarily on domain-general executive brain regions. *eLife* 9 (dec 2020), e58906. <https://doi.org/10.7554/eLife.58906>
- [20] David Joyner, Ryan Arrison, Mehnaz Ruksana, Evi Salguero, Zida Wang, Ben Wellington, and Kevin Yin. 2019. From Clusters to Content: Using Code Clustering for Course Improvement. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education (Minneapolis, MN, USA) (SIGCSE ’19)*. Association for Computing Machinery, New York, NY, USA, 780–786. <https://doi.org/10.1145/3287324.3287459>

- [21] Juho Kim, Elena I. Glassman, Andrés Monroy-Hernández, and Meredith Ringel Morris. 2015. RIMES: Embedding interactive multimedia exercises in lecture videos. In *Proceedings of the 33rd annual ACM conference on human factors in computing systems*. 1535–1544.
- [22] Sean Kross and Philip J. Guo. 2019. Practitioners teaching data science in industry and academia: Expectations, workflows, and challenges. In *Proceedings of the 2019 CHI conference on human factors in computing systems*. 1–14.
- [23] Adrian Kuhn, Stéphane Ducasse, and Tudor Gîrba. 2007. Semantic clustering: Identifying topics in source code. *Information and Software Technology* 49, 3 (2007), 230–243. <https://doi.org/10.1016/j.infsof.2006.10.017> 12th Working Conference on Reverse Engineering.
- [24] Leland McInnes, John Healy, Steve Astels, et al. 2017. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.* 2, 11 (2017), 205.
- [25] Pedro Orvalho, Mikolás Janota, and Vasco M. Manquinho. 2022. InvAASSTCluster: On Applying Invariant-Based Program Clustering to Introductory Programming Assignments. *abs/2206.14175* (2022).
- [26] Jeffrey M. Perkel. 2018. Why Jupyter is data scientists’ computational notebook of choice. *Nature* 563, 7732 (2018), 145–147.
- [27] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2019. A large-scale study about quality and reproducibility of jupyter notebooks. In *2019 IEEE/ACM 16th international conference on mining software repositories (MSR)*. IEEE, 507–517.
- [28] Lutz Prechelt and Guido Malpohl. 2003. Finding Plagiarisms among a Set of Programs with JPlag. *Journal of Universal Computer Science* 8 (03 2003).
- [29] Bernadette M. Randles, Irene V. Pasquetto, Milena S. Golshan, and Christine L. Borgman. 2017. Using the Jupyter Notebook as a Tool for Open Science: An Empirical Study. In *2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. 1–2. <https://doi.org/10.1109/JCDL.2017.7991618>
- [30] Adam Rule, Aurélien Tabard, and James D. Hollan. 2018. Exploration and explanation in computational notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [31] D.A. Scanlan. 1989. Structured flowcharts outperform pseudocode: an experimental comparison. *IEEE Software* 6, 5 (1989), 28–36. <https://doi.org/10.1109/52.35587>
- [32] Jeremy Singer. 2020. Notes on notebooks: is Jupyter the bringer of jollity?. In *Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Virtual, USA) (Onward! 2020)*. Association for Computing Machinery, New York, NY, USA, 180–186. <https://doi.org/10.1145/3426428.3426924>
- [33] Weisong Sun, Yun Miao, Yuekang Li, Hongyu Zhang, Chunrong Fang, Yi Liu, Gelei Deng, Yang Liu, and Zhenyu Chen. 2024. Source Code Summarization in the Era of Large Language Models. *arXiv:2407.07959 [cs.SE]* <https://arxiv.org/abs/2407.07959>
- [34] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [35] April Yi Wang, Anant Mittal, Christopher Brooks, and Steve Oney. 2019. How data scientists use computational notebooks for real-time collaboration. *Proceedings of the ACM on Human-Computer Interaction* 3, CSCW (2019), 1–30.
- [36] Doris Xin, Litian Ma, Shuchen Song, and Aditya G. Parameswaran. 2018. How Developers Iterate on Machine Learning Workflows - A Survey of the Applied Machine Learning Literature. *ArXiv abs/1803.10311* (2018). <https://api.semanticscholar.org/CorpusID:4378544>
- [37] Litao Yan, Alyssa Hwang, Zhiyuan Wu, and Andrew Head. 2024. Ivie: Lightweight Anchored Explanations of Just-Generated Code. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (Honolulu, HI, USA) (CHI ’24)*. Association for Computing Machinery, New York, NY, USA, Article 140, 15 pages. <https://doi.org/10.1145/3613904.3642239>
- [38] Ashley Ge Zhang, Yan Chen, and Steve Oney. 2023. VizProg: Identifying Misunderstandings By Visualizing Students’ Coding Progress. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (Hamburg, Germany) (CHI ’23)*. Association for Computing Machinery, New York, NY, USA, Article 596, 16 pages. <https://doi.org/10.1145/3544548.3581516>
- [39] Ashley Ge Zhang, Xiaohang Tang, Steve Oney, and Yan Chen. 2024. CFlow: Supporting Semantic Flow Analysis of Students’ Code in Programming Problems at Scale. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale (Atlanta, GA, USA) (L@S ’24)*. Association for Computing Machinery, New York, NY, USA, 188–199. <https://doi.org/10.1145/3657604.3662025>
- [40] Feng Zhang, Guofan Li, Cong Liu, and Qian Song. 2020. Flowchart-Based Cross-Language Source Code Similarity Detection. *Scientific Programming* 2020, 1 (2020), 8835310.

A RESEARCH METHODS

A.1 Classification labels

We defined a set of 8 labels, inspired by the 1st-layer-labels of the Code4ML’s taxonomy tree. The labels are quite self-explanatory but here is an overview:

- (1) **Imports and Environment:** this class is for cells which in majority have "import" statements or setup environment variables.
- (2) **Data Extraction:** code in this class are for extracting data from any source, like CSV files or databases for instance.
- (3) **Data Transform:** this class is for any code cells which perform any kind of transformation on the data, whether it be mathematical operations or any data preprocessing.

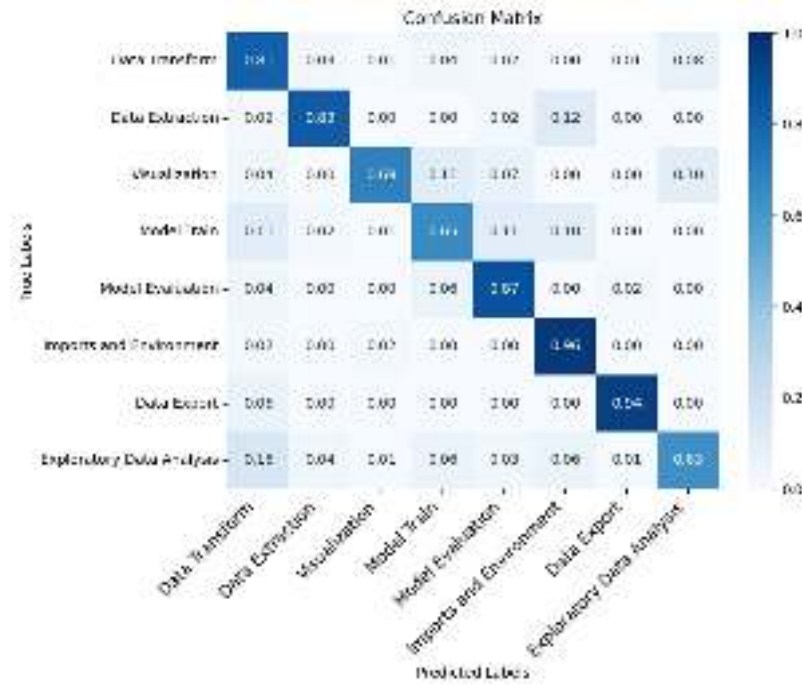


Fig. 5. Confusion matrix displaying the classification performance across multiple data science stages, with the color intensity representing the frequency of predicted vs. true labels.

- (4) **Exploratory Data Analysis:** code cells in this class mainly display data to the viewer presented in a specific format, whether it be just printing out variables or displaying a Pandas dataframe for instance. It is for the user to gain explore and gain knowledge about the data.
- (5) **Visualization** much like EDA, this class is for cells which present data, here more visually, like plots or images.
- (6) **Model Training:** like the name suggests it, this class is for the actual training of the model.
- (7) **Model Evaluation:** this is for code cells which evaluate the performance of the trained model, using whatever metrics are relevant for the user.
- (8) **Data Export:** this class is for gathering the final results of the data science pipeline and optionally saving or exporting them to a specific format.

A.2 Competition questions

During the evaluation, we asked participants to answer two set of questions about the Kaggle competitions we chose.

A.2.1 Natural Language Processing with Disaster Tweets.

- How did participants handle misspellings and informal language in the tweets?
- What is the most commonly used visualization for evaluating model performance in disaster tweets analysis?
- What types of machine learning models are predominantly used for predicting tweets?
- What hyperparameter tuning technique is the most used to optimize model performance?

- What method was the most used to evaluate the model's performance on the disaster tweets dataset?

A.2.2 *Titanic - Machine Learning from Disaster.*

- What are the most common features used by students to predict survival on the Titanic?
- How do participants handle missing data in the Titanic dataset?
- What types of machine learning models are predominantly used for predicting survival?
- How do participants evaluate model performance on the test set?
- How do participants select the best models?

A.3 NASA TLX Questions

During the evaluation, we asked participants to answer questions after each task, adapted from the NASA TLX Index [16].

The questions were:

- How mentally demanding was the task?
- How hurried or rushed was the pace of the task?
- How successful were you in accomplishing what you were asked to do?
- How hard did you have to work to accomplish your level of performance?
- How frustrated were you when completing the task?

Question	p-value	Baseline Mean (Std)	Experiment Mean (Std)
Mental demand	0.025984	3.90 (0.718185)	3.20 (0.951453)
Hurriedness	0.281609	3.50 (1.147079)	3.05 (1.145931)
Success	0.213346	2.95 (1.145931)	3.35 (0.933302)
Effort	0.041274	3.60 (0.820783)	2.90 (0.852242)
Frustration	0.035522	3.30 (1.454575)	2.30 (1.218282)

Table 3. Wilcoxon Test Results for NASA TLX Questions

A.4 Post-study open questions

At the end of the study, we asked some open-ended questions to the participants. The questions being:

- What did you like most about using the tool?
- What improvements would you suggest for the tool?
- Do you have any additional comments or feedback about the experience?
- Would you consider using this tool in the future? What scenarios would this tool be useful for?

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009